CS 182 - HW 8 - Jie Wen

1. SSM Convolution Kernel

(a) Convolution Kernel and the Output Equation

Starting from $x_0 = 0$, unroll the state equation:

$$x_k = \sum_{i=0}^{k-1} A^{k-1-i} B u_i$$

Substitute into the output equation:

$$y_k = Cx_k + Du_k = C\left(\sum_{i=0}^{k-1} A^{k-1-i} B u_i\right) + Du_k = \sum_{i=0}^{k-1} CA^{k-1-i} B u_i + Du_k$$

Change the index by letting $l = k - i$, so $i = k - l$, and when $i = 0$, $l = k$; when $i = k - 1$, $l = 1$. The sum becomes:

$$y_k = Du_k + \sum_{l=1}^{k} CA^{l-1} B u_{k-l} = \sum_{l=0}^{k} K_l u_{k-l}$$

where $u_{k-l} = 0$ for $k - l < 0$, and the kernel is:

$$K_l = \begin{cases} D & l = 0 \\ CA^{l-1}B & l \geq 1 \end{cases}$$

The sum extends to L with zero-padding for indices outside the sequence.

(b) Concrete Examples

i. *Scalar Case*

Assume $\delta = 0$ (consistent with the second example and common in SSMs). Then $K_0 = 0$, and for $l \geq 1$,

$$K_l = \gamma \alpha^{l-1} \beta = 1.5 \times 0.8^{l-1} \times 1 = 1.5 \times 0.8^{l-1}$$

Up to $L = 4$:

- $K_0 = 0$
- $K_1 = 1.5 \times 0.8^0 = 1.5$
- $K_2 = 1.5 \times 0.8^1 = 1.2$
- $K_3 = 1.5 \times 0.8^2 = 0.96$
- $K_4 = 1.5 \times 0.8^3 = 0.768$

ii. *2D Case*

Here $D = 0$, so $K_0 = 0$, and for $l \geq 1$, $K_l = CA^{l-1}B$.

Compute matrix powers:

$$A = \begin{pmatrix} 0.7 & 0.1 \\ 0.2 & 0.6 \end{pmatrix}, \quad A^2 = \begin{pmatrix} 0.51 & 0.13 \\ 0.26 & 0.38 \end{pmatrix}$$

Compute $AB = \begin{pmatrix} 0.7 \\ 0.2 \end{pmatrix}$, then $C(AB) = 0.7$. Next, $A^2 = A \cdot A$ via matrix multiplication (row-by-column dot products), then $A^2B = \begin{pmatrix} 0.51 \\ 0.26 \end{pmatrix}$, and $C(A^2B) = 0.51$.

Up to $L = 3$:

- $K_0 = 0$
- $K_1 = CB = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 1$
- $K_2 = CAB = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 0.7 \\ 0.2 \end{pmatrix} = 0.7$
- $K_3 = CA^2B = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 0.51 \\ 0.26 \end{pmatrix} = 0.51$

The kernel captures the impulse response: If the input is an impulse ($u_0 = 1$, others 0), then $y_k = K_k$, showing how the system's state evolves and affects the output over time from a single input perturbation.

(c) Efficient Computation with Convolutions

With known K, the convolution $y_k = \sum_{l=0}^{L} K_l u_{k-l}$ (zero-padded) can be fully parallelized across the $L+1$ outputs, as each y_k is independent. Using direct summation, each y_k requires a parallel sum of $O(L)$ terms, which has a critical path of $O(\log L)$ via parallel reduction (assuming unlimited processors). Alternatively, via FFT (pad to $O(L)$, FFT both sequences, pointwise multiply, inverse FFT), the critical path is also $O(\log L)$ due to the butterfly network in FFT.

For naive direct computation from unrolled recursion ($y_k = Du_k + \sum_{i=0}^{k-1} CA^{k-1-i}Bu_i$), it's equivalent to direct convolution, so same $O(\log L)$ critical path. However, computing via the sequential recursion ($x_{k+1} = Ax_k + Bu_k$, $y_k = Cx_k + Du_k$) has a critical path of $O(L)$, as each step depends on the previous state.

(d) Efficient Kernel Computation

To compute $K = \{K_l\}_{l=0}^{L}$, where $K_0 = D$ and $K_l = CA^{l-1}B$ for $l \geq 1$:

- Naive: Compute the sequence $v_0 = B$, $v_l = Av_{l-1}$ for $l = 1$ to L, then $K_l = Cv_{l-1}$ for $l \geq 1$. This is sequential with critical path $O(L)$.

Strategies to parallelize (assuming $L = 2^N$):

- Use parallel prefix computation on matrix multiplications. Consider the sequence of L copies of A; the prefix products give $A^1, A^2, \ldots, A^L$. Parallel prefix has critical path $O(\log L)$ using the Ladner-Fischer or similar algorithm (doubling spans in each round). Then, multiply each A^k by B and C in parallel (additional $O(1)$ assuming fixed dimensions).

This reduces the critical path to $O(\log L)$.

(e) Adding structure to A

If A is a diagonal matrix,

$$A = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}$$

then,

$$A^{l-1} = \begin{pmatrix} \lambda_1^{l-1} & 0 & \cdots & 0 \\ 0 & \lambda_2^{l-1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^{l-1} \end{pmatrix}$$

so

$$K_l = CA^{l-1}B = \sum_{j=1}^{n} c_j \lambda_j^{l-1} b_j$$

(where c_j, b_j are components if vectorized, or matrix form).

Leverage this: Compute each λ_j^{l-1} independently across j. For each eigenvalue, the sequence $\{\lambda_j^l\}_{l=0}^{L}$ is a geometric series, computable via parallel prefix multiplication per sequence (critical path $O(\log L)$ per j). Then, sum over j in parallel for each l (critical path $O(\log n)$). Overall critical path $O(\log L + \log n)$, and operations are scalar, avoiding full matrix multiplications.

(f) Diagonal-plus-low-rank (DPLR) structure

With $A = I_n + pp^T$, let $s = p^T p$. Then,

$$A^k = I_n + pp^T \cdot \frac{(1+s)^k - 1}{s} \quad (s \neq 0)$$

(For $s = 0$, $A = I_n$, trivial.) Derive this via binomial expansion and properties of rank-1 matrices: $(pp^T)^j = s^{j-1}pp^T$ for $j \geq 1$, leading to the scalar coefficient sum.

Thus, for $l \geq 1$,

$$K_l = CA^{l-1}B = CB + \left(\frac{(1+s)^{l-1} - 1}{s}\right)(Cp)(p^T B)$$

Leverage this: The matrix power has a closed scalar form. Compute the sequence $\{(1+s)^{l-1}\}_{l=1}^{L+1}$ via parallel prefix on scalar multiplications (critical path $O(\log L)$). Then, compute each K_l in parallel using the formula (additional $O(1)$, scalar and dot products). This avoids matrix exponentiation, with operations independent of n in complexity for the powers, making it highly efficient for large n.

---

2. Coding SSM Forward

CPU Implementation

(a) Recurrent (time-unrolled) forward pass

The computation is

$$h_{t+1} = Wh_t + Ux_t + b, \qquad h_0 = 0,$$

for $t = 0, \ldots, T-1$. In code, $s_t = Ux_t + b$ is precomputed for all t, then the process iterates:

- Initialize $h_0 = \mathbf{0}$.
- For each t: $h_{t+1} = Wh_t + s_t$.
- Store all h_{t+1} as (N, T, H).

This is exactly what the `unrolled_ssm_forward` function does (precompute $s = x@U.T + b$, loop over time, multiply by $W^\top$ once per step, and accumulate).

(b) Convolution-based forward pass

The solution can be written as a causal convolution of the projected inputs with powers of W:

$$h_t = \sum_{\ell=0}^{t-1} W^\ell (Ux_{t-1-\ell} + b).$$

Define a kernel $K \in \mathbb{R}^{H \times H \times T}$ with $K[:, :, \ell] = W^\ell$ for $\ell = 0, \ldots, T-1$. After projecting inputs $s_t = Ux_t + b$ and transposing to (N, H, T), a 1-D causal cross-correlation is applied using `conv1d` with weight K (time-reversed in the implementation to match PyTorch's cross-correlation). Only the first T outputs are kept to preserve causality. The `make_conv_kernel` function fills $\{W^\ell\}$ using a doubling pattern and the `conv_ssm_forward` function performs the convolution exactly as above.

(c) Runtime comparison & explanation

Empirics (plot for $H = 4$): the unrolled method is consistently faster once T grows beyond very small sizes, and its runtime scales roughly linearly with T. The convolution method grows much faster with T and becomes significantly slower.

Why this happens here:

- **Unrolled**: Each step does one $(N \times H) \times (H \times H)$ multiply plus a vector add → cost $\Theta(N, H^2)$ per step → $\Theta(N, T, H^2)$ overall (plus the one-time (Ux) projection $\Theta(N, T, D, H)$).
- **Conv (direct)**: With kernel size T and output length T, 1-D conv cost is roughly $\Theta(N, H_{\text{out}}, H_{\text{in}}, T, \text{kernel}) = \Theta(N, H^2, T^2)$. Building K also costs $\Theta(T, H^3)$ multiplications of $H \times H$ matrices. Thus for general dense W, the conv path is **quadratic in** T (and heavy in H) unless an FFT-based convolution is used (which could reduce the time part to $\Theta(N, H^2, T \log T)$, but would still pay $\Theta(T, H^3)$ to form all W^ℓ and $\Theta(T, H^2)$ memory).
- **Memory**: Conv needs to store the whole kernel $K \in \mathbb{R}^{H \times H \times T} \to \Theta(T, H^2)$ memory, versus the unrolled method which only needs $\Theta(N, H)$ working memory (plus $\Theta(N, T, H)$ to keep outputs).

Bottom line: For dense W and moderate T, the unrolled implementation dominates on CPU (and often on GPU) because it's linear in T, while the direct conv is quadratic in T and allocates a big kernel. Convolution only starts to make sense with **structured** W (e.g., diagonal/low-rank), **FFT convolution**, or when massive parallelism can be exploited to amortize kernel costs.

Notebook Q&A

Question 1

Maximum absolute difference between the two implementations (from the printout):

$$\boxed{5.960464477539063 \times 10^{-8}}$$

(numerical noise ≈ single-precision eps).

Question 2

Operation count (recurrence)
Ignoring constant factors and additions:

- Per time step: $(N \times H) \times (H \times H) \Rightarrow \Theta(N, H^2)$.
- Across T steps: $\boxed{\Theta(N, T, H^2)}$.
  Plus the input projection once: $x@U^\top$ costs $\Theta(N, T, D, H)$.

Question 3

Operation count (convolution)

- Building K with all powers W^ℓ: $\boxed{\Theta(T, H^3)}$.
- Direct 1-D conv with H in-channels, H out-channels, kernel size T, output length T:
  $\boxed{\Theta(N, H^2, T^2)}$.
  (If using FFT conv it can be $\Theta(N, H^2, T \log T)$, but the code used direct `conv1d`.)

Question 4

Trade-offs

- **Unrolled**:

- Time $\Theta(N, T, H^2)$ (linear in T), low extra memory.
- Simple, numerically stable, no big kernel tensor.

- **Conv (as implemented)**:
  - Time $\Theta(N, H^2, T^2) + \Theta(T, H^3)$ (quadratic in T), memory $\Theta(T, H^2)$.
  - Potential advantages only if (i) direct conv is replaced with **FFT**, (ii) W is **structured** so K is cheap/compact, or (iii) massive parallelism is exploited.

Question 5

Observed from the plot ($H = 4$) & explanation

- The **unrolled** curve grows roughly **linearly in** T and stays faster for medium/large T.
- The **conv** curve grows **super-linearly (~T^2)**, quickly overtaking unrolled in cost.
  This matches the complexity: unrolled is $\Theta(T)$ in T while the direct convolution is $\Theta(T^2)$. As H increases, both get slower; the conv approach also suffers from building/storing K ((T, H^2) memory, (T, H^3) build time), further widening the gap unless specialized structure/FFT is used.

GPU Implementation

(d) GPU forward pass

Both implementations were ported to CUDA by setting `torch.set_default_device('cuda')` and ensuring all tensors (inputs, params, and intermediate buffers) live on the GPU. For the convolutional path, **causal** convolution is implemented correctly (PyTorch `conv1d` is *cross-correlation*, so the kernel is flipped along time and the input is **left-padded** by $T - 1$ before calling `F.conv1d`), which preserves output length T. PyTorch's documentation notes that `Conv1d` uses cross-correlation and that `padding` is applied **on both sides**, hence the need for the flip+left-pad trick for causal behavior.

(e) Runtime comparison (unrolled vs. conv on GPU)

From the first plot ("Runtime vs T, H=4"):

- **Conv** is consistently faster than **Unrolled** across all T, with a widening gap as T grows.
- The conv curve is much flatter vs. the unrolled curve (which grows quickly with T).

Why this happens on GPU

- The **unrolled** method is inherently **sequential in time** (each step depends on the previous), so it cannot be parallelized across T and ends up launching many small matrix ops with poor GPU utilization.
- The **conv** method turns the recurrence into one large batched convolution that maps well to cuDNN's highly optimized GPU kernels (implicit-GEMM / transform-based algorithms), giving excellent throughput, especially as T grows.

(f) Diagonal W + depth-wise conv

The second plot ("Runtime vs T, H=4 diagonal weights") shows:

- **Diag-Conv** (depth-wise) is **orders of magnitude faster** and almost flat vs. T.
- **Unrolled** still scales linearly in T with a much larger slope.

Why this is faster

- Constraining W to be **diagonal** removes cross-channel mixing, so each hidden channel updates independently. Implementing the convolution as **depth-wise** (`groups=H`) slashes compute from $O(H^2T)$ to $O(HT)$ and maps to efficient grouped kernels. This is the same rationale as **depthwise separable convolutions** widely used to cut FLOPs.
- On modern GPUs, depth-wise/grouped convs are supported in cuDNN/optimized paths (with performance nuances by dtype/device), but generally remain very efficient for this pattern.

Notebook Q&A

Question 6

Observation. As T increases (and for larger H), the **convolutional** approach remains substantially faster than the **unrolled** approach. The gap grows with T. This **matches the trend** seen on CPU but is **even more pronounced on GPU**, where the conv path benefits from cuDNN's high-throughput kernels while the unrolled path is bottlenecked by time-step serial dependency and many small launches.

Reasoning.

- Unrolled: strictly sequential across time → poor parallelism over T.
- Conv: one (or a few) large convolutions → excellent GPU utilization using cuDNN (implicit-GEMM/transform-based).

So: **same qualitative trend as CPU**, but the **crossover occurs earlier** and the **conv slope is flatter** on GPU due to better kernel efficiency and batching.

Question 7

Observation. With **diagonal** W, the **depth-wise conv** implementation is dramatically faster than unrolled and nearly flat vs. T in the plot; the speedup is larger than in the unstructured case.

How it differs from the unstructured case.

- Unstructured W: conv still does $O(H^2T)$ work (mixing across channels).
- **Diagonal** W: each channel is independent → **depth-wise** conv with `groups=H` → **computational cost drops to** $O(HT)$ and memory traffic simplifies. This mirrors the well-known savings of **depthwise separable convolutions** in CNNs.

Why GPU loves this setting. Grouped/depth-wise kernels are streamlined and avoid large H×H multiplies; cuDNN's convolution backends (implicit-GEMM/transform-based) execute these patterns efficiently, so runtime becomes dominated by highly parallel per-channel operations rather than sequential time steps.

---

3. Self-Supervised Linear Purification

(a)

i.

Encoder	Reconstruction Loss	Regularization Loss
α	0	2
β	0.001	1

To compute the reconstruction loss for $W^{(\alpha)}$, note that $W^{(\alpha)} = I$, so $X - W^{(\alpha)}X = 0$ and $\|X - W^{(\alpha)}X\|_F^2 = 0$. The regularization loss is $\|W^{(\alpha)}\|_F^2 = 1^2 + 1^2 = 2$.

For $W^{(\beta)}$, the reconstruction loss is given as 0.001. To verify, compute $W^{(\beta)}X$, which sets the second row to zero, so $X - W^{(\beta)}X$ has zeros in the first row and the second row of X in the second row. The Frobenius norm squared is $0.02^2 + (-0.01)^2 + 0.01^2 + (-0.02)^2 = 0.0004 + 0.0001 + 0.0001 + 0.0004 = 0.001$. The regularization loss is $\|W^{(\beta)}\|_F^2 = 1^2 + 0^2 = 1$.

ii.

The total loss for $W^{(\alpha)}$ is $\mathcal{L}_2(W^{(\alpha)}; \mathbf{X}, \lambda) = 0 + \lambda \cdot 2 = 2\lambda$.
The total loss for $W^{(\beta)}$ is $\mathcal{L}_2(W^{(\beta)}; \mathbf{X}, \lambda) = 0.001 + \lambda \cdot 1 = 0.001 + \lambda$.
Set $2\lambda > 0.001 + \lambda$, which simplifies to $\lambda > 0.001$.
To solve, subtract λ from both sides: $\lambda > 0.001$.

(b)

i.

$$\widehat{W} = [U] \begin{bmatrix} \frac{\sigma_1^2}{\sigma_1^2+\lambda} & & & \\ & \frac{\sigma_2^2}{\sigma_2^2+\lambda} & & \\ & & \ddots & \\ & & & \frac{\sigma_m^2}{\sigma_m^2+\lambda} \end{bmatrix} [U^\top]$$

ii.

The objective is $\mathcal{L}_2(W; \mathbf{X}, \lambda) = \|X - WX\|_F^2 + \lambda\|W\|_F^2$.
This is equivalent to a multivariate ridge regression problem where the target $Y = X$ and the features are also X. The closed-form solution for such a ridge regression is $\widehat{W} = XX^\top(XX^\top + \lambda I)^{-1}$.

Given the SVD $X = U\Sigma V^\top$, compute $XX^\top = U\Sigma V^\top V\Sigma U^\top = U\Sigma^2 U^\top$, since $V^\top V = I_m$.
Then $XX^\top + \lambda I = U(\Sigma^2 + \lambda I)U^\top$.
The inverse is $(XX^\top + \lambda I)^{-1} = U(\Sigma^2 + \lambda I)^{-1}U^\top$.
Thus, $\widehat{W} = U\Sigma^2 U^\top \cdot U(\Sigma^2 + \lambda I)^{-1}U^\top = U\Sigma^2(\Sigma^2 + \lambda I)^{-1}U^\top$.
Since Σ^2 and $\Sigma^2 + \lambda I$ are diagonal, this is U diag $\left(\frac{\sigma_i^2}{\sigma_i^2+\lambda}\right)U^\top$.

Alternatively, using the invariance of the Frobenius norm under orthogonal transformations, define $W' = U^\top WU$. The loss becomes $\|\Sigma - W'\Sigma\|_F^2 + \lambda\|W'\|_F^2$.
This decouples into m independent ridge regression problems along each singular direction, yielding the same diagonal form for W' and thus for $\widehat{W}$.

(c)

The scaling factor along the i-th singular direction is $\alpha_i = \frac{\sigma_i^2}{\sigma_i^2+\lambda}$.
Preserving at least 80% for the first 3 directions requires $\alpha_i \geq 0.8$ for $i = 1, 2, 3$.
This simplifies to $\lambda \leq 0.25\sigma_i^2$ for each $i = 1, 2, 3$.
The most restrictive is for $\sigma_3 = 4$, giving $\lambda \leq 0.25 \cdot 16 = 4$.

Attenuating the remaining to at most 50% requires $\alpha_i \leq 0.5$ for $i = 4, \ldots, 8$.
This simplifies to $\lambda \geq \sigma_i^2$ for each $i = 4, \ldots, 8$.

The most restrictive is for $\sigma_4 = 1$, giving $\lambda \geq 1$.

Thus, the condition holds for $\lambda \in [1, 4]$.
To solve for $\alpha_i \geq 0.8$: $\frac{\sigma_i^2}{\sigma_i^2+\lambda} \geq 0.8 \implies \sigma_i^2 \geq 0.8(\sigma_i^2 + \lambda) \implies 0.2\sigma_i^2 \geq 0.8\lambda \implies \lambda \leq 0.25\sigma_i^2$.
For $\alpha_i \leq 0.5$: $\frac{\sigma_i^2}{\sigma_i^2+\lambda} \leq 0.5 \implies \sigma_i^2 \leq 0.5(\sigma_i^2 + \lambda) \implies 0.5\sigma_i^2 \leq 0.5\lambda \implies \lambda \geq \sigma_i^2$.

---

4. Ridge-Attention

(a)

$$m' = \frac{1}{n+1}\sum_{i=1}^{n+1} x_i = \frac{1}{n+1}\Big(nm + x_{n+1}\Big) = \frac{nm + x_{n+1}}{n+1}.$$

(b)
Using $A = [x_1^\top; \ldots; x_n^\top]$ and $y = [y_1; \ldots; y_n]$,

$$A^\top A = \sum_{i=1}^{n} x_i x_i^\top, \qquad A^\top y = \sum_{i=1}^{n} x_i y_i.$$

Thus

$$(A^\top A + \lambda I) = \lambda I + \sum_{i=1}^{n} x_i x_i^\top, \quad A^\top y = \sum_{i=1}^{n} x_i y_i,$$

consistent with the ridge solution $w_* = (A^\top A + \lambda I)^{-1} A^\top y$.

(c) Computational cost of **non-causal ridge self-attention** (with shared n keys/values for all n queries, $d < n$):

- Form $K^\top K$: $O(nd^2)$.
- Invert $K^\top K + \lambda I$: $O(d^3)$.
- Form $K^\top V$: $O(nd^2)$.
- Multiply $(K^\top K + \lambda I)^{-1}(K^\top V)$: $O(d^3)$.
- For all n queries, compute $q^\top W$ (each is $O(d^2)$): total $O(nd^2)$.

Total $O(nd^2 + d^3)$, dominated by $O(nd^2)$ when $d \ll n$.
Answer: $\boxed{O(nd^2)}$.

(d) Backprop will flow through every differentiable piece of the formula

$$\hat{y} = q^\top (K^\top K + \lambda I)^{-1} K^\top V$$

so gradients pass to **λ, keys, values, and queries**.
Answer: check **all four**. (Ridge solution is differentiable in all of these under typical conditions.)

(e) Using Sherman–Morrison,

$$(M + uv^\top)^{-1} = M^{-1} - \frac{(M^{-1}u)(v^\top M^{-1})}{1 + v^\top M^{-1} u}.$$

Given M^{-1}, compute $a = M^{-1}u$ ($O(d^2)$), $b = v^\top M^{-1}$ or $M^{-1}v$ ($O(d^2)$), the scalar $v^\top a$ ($O(d)$), and the outer product ab ($O(d^2)$). Net cost: $O(d^2)$.
Answer: $\boxed{O(d^2)}$.

(f) Efficient causal ridge self-attention (online update).

Maintain, for step t:

- $S_t = K_{1:t}^\top K_{1:t} + \lambda I \in \mathbb{R}^{d\times d}$ and its inverse $P_t = S_t^{-1}$.
- $T_t = K_{1:t}^\top V_{1:t} \in \mathbb{R}^{d\times d}$.

Initialization: $S_0 = \lambda I, P_0 = \lambda^{-1} I, T_0 = 0$.

On receiving (k_t, v_t):

1. **Rank-1 update for S_t:** $S_t = S_{t-1} + k_t k_t^\top$.
   Update P_t from P_{t-1} via Sherman–Morrison with $u = v = k_t$:
   $$P_t = P_{t-1} - \frac{P_{t-1} k_t k_t^\top P_{t-1}}{1 + k_t^\top P_{t-1} k_t} \quad (O(d^2)).$$
2. **Update T_t:** $T_t = T_{t-1} + k_t v_t^\top$ (a dyad, $O(d^2)$).
3. **Return output for the current query q_t:**
   Compute $u_t^\top = q_t^\top P_t$ ($O(d^2)$), then $\hat{y}_t = u_t^\top T_t$ ($O(d^2)$).

This yields **per-step** $O(d^2)$, hence total $O(nd^2)$ for all n positions, without recomputing any $d \times d$ inverse from scratch.

(g) "Attention weights" in ridge-attention.
For a query q, the multi-output ridge prediction is

$$\hat{Y}(q) = q^\top (K^\top K + \lambda I)^{-1} K^\top V.$$

This can be written as a **linear combination of the value rows**:

$$\hat{Y}(q) = \alpha(q)^\top V, \quad \alpha(q) = K(K^\top K + \lambda I)^{-1} q \in \mathbb{R}^n.$$

So the (possibly negative) "weights" on each value vector v_i are the entries of $\alpha(q)$.

---

5. Homework Process and Study Group

(a)

1. Lecture Notes
2. **State-Space Models** (MIT OpenCourseWare) - Core material on state-space representations, a foundational concept in control theory and signal processing that has heavily influenced modern sequence models.
3. **High-Dimensional Regression: Ridge** (UC Berkeley Stats Lecture Notes) - A clear and modern educational resource explaining the theory and derivation of Ridge Regression, a cornerstone of regularized models.
4. **Recurrent neural network** (Wikipedia) - A central, high-quality, and accessible overview of a major neural network architecture, its variants (like LSTMs), and its applications.
5. **FFT Convolution** (Analog Devices) - A practical and theoretical guide to using the Fast Fourier Transform for efficient convolution, a core concept in signal processing and deep learning.
6. **The Sherman–Morrison–Woodbury formula** - A fundamental linear algebra identity that is crucial for efficient matrix inversion and updates in algorithms like recursive least squares and Gaussian processes.
7. **PyTorch Documentation: Conv1d** (PyTorch Development Team) - Official API documentation for a fundamental building block in a major deep learning framework, essential for practical implementation.
8. **NVIDIA: Optimizing Convolutional Layers User's Guide** (NVIDIA) - An official guide from the leading hardware provider on how to optimize the performance of convolutional layers, a critical aspect of modern deep learning.
9. **Diagonalizable matrix** (Wikipedia) - A comprehensive explanation of matrix diagonalization, a key concept in linear algebra with applications in many areas of machine learning and signal processing.
10. **State Space Formulation • Markov Parameters** (University of West Bohemia) - A presentation on state-space models and Markov parameters, providing a concise overview of these important concepts in linear systems theory.
11. **How to derive the ridge regression solution?** (Cross Validated) - A community-driven explanation of the derivation of the ridge regression solution, offering multiple perspectives on this fundamental machine learning technique.
12. **How to calculate moving average without keeping the count and data total?** (Stack Overflow) - A practical discussion of different methods for calculating moving averages, a common task in time series analysis and signal processing.
13. **Optimizing Convolutional Layers** (NVIDIA) - A detailed guide on optimizing convolutional layers for deep learning applications, covering topics such as tensor layouts, implicit GEMM, and the impact of various convolution parameters on performance.
14. **Depthwise Separable Convolution - an overview** (ScienceDirect) - An overview of depthwise separable convolution, a type of convolutional layer that is more efficient than standard convolution, with applications in mobile and embedded vision applications.
15. **Sherman–Morrison formula** (Wikipedia) - An explanation of the Sherman-Morrison formula, a tool for updating the inverse of a matrix after a rank-1 update, which has applications in optimization and numerical linear algebra.

(b)

None

(c)

1.5 day